KELOMPOK 3 · PEMROGRAMAN BERORIENTASI OBJEK

Naskah Presentasi

Sistem Manajemen Data Pasien Penyakit Mata

Naskah lisan untuk 41 slide presentasi HTML. Teksnya sama persis dengan catatan pembicara di file presentasi (tekan N saat presentasi dibuka).

No	Pembicara	Slide	Kata	Waktu baca	Waktu tampil
1	**Riswan**	1–5	333	± 2 menit 45 detik	± 3 menit 30 detik
2	**Yus Aidil**	6–9	327	± 2 menit 45 detik	± 3 menit 25 detik
3	**Fahmi**	10–13	326	± 2 menit 45 detik	± 3 menit 25 detik
4	**Diva Nurislamia Agfa**	14–18	392	± 3 menit 15 detik	± 4 menit 5 detik
5	**Rosa Ramadani**	19–22	291	± 2 menit 25 detik	± 3 menit
6	**Sofiang**	23–26	298	± 2 menit 30 detik	± 3 menit 5 detik
7	**Revalinda**	27–30	321	± 2 menit 40 detik	± 3 menit 20 detik
8	**Andi Rezky Nabila**	31–34	371	± 3 menit 5 detik	± 3 menit 50 detik
9	**Alfira Yunita Syam**	35–41	496	± 4 menit 10 detik	± 5 menit 10 detik
	Total	1–41	**3155**	**± 26 menit 20 detik**	**± 32 menit 50 detik**

Waktu baca dihitung 120 kata per menit. Waktu tampil ditambah ±25% untuk jeda, klik, dan menunjuk kode; totalnya ± 33 menit.

Cara membaca naskah

- KLIK → = ganti ke slide berikutnya (panah kanan, spasi, atau klik roda mouse).
- (tunjuk: baris 14) = arahkan pointer atau kursor ke bagian yang disebut.
- **Huruf tebal** = kata kunci, ucapkan lebih jelas dan pelan.
- *(jeda)* = berhenti sejenak, biasanya sebelum kalimat estafet ke pembicara berikutnya.
- Tabel **Penjelasan kode** di bawah naskah menjelaskan fungsi setiap baris kode yang tampil di slide. Nomor barisnya sama persis dengan file di folder `program`.
- Semua angka dan output berasal dari program yang **benar-benar dijalankan** dengan data asli.

Tips tampil

- Buka file presentasi HTML di Chrome atau Edge, lalu tekan **F11** agar layar penuh.
- Tekan **N** untuk membuka naskah slide yang sedang tampil (untuk latihan). Tutup lagi dengan N sebelum tampil di proyektor.
- Klik nama anggota di slide **Sembilan bagian, sembilan pembicara** untuk melompat ke bagiannya, berguna saat sesi tanya jawab.
- Pahami alurnya, jangan menghafal kata per kata. Pakai **kata kunci** di tiap bagian sebagai pegangan, dan latihan minimal dua kali penuh dengan stopwatch, termasuk pergantian pembicara.
- Output program memakai titik sebagai desimal (`57.9`); bacakan dengan format Indonesia: “lima puluh tujuh koma sembilan”.

BAGIAN 1 | RISWAN

Latar Belakang & Struktur Project

Slide 1–5 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 45 detik • 333 kata

Kata kunci: Sistem Manajemen Data Pasien Penyakit Mata · class dan object · enkapsulasi · inheritance · 3.358 pasien · 7 kelompok penyakit · 9 class · kata kunci · 6.392 foto dari 3.358 pasien · terstruktur, aman, dan mudah dicari · hierarki class · program · sepuluh file · full_df.csv · python main.py · menu interaktif

SLIDE 1 Sistem Manajemen Data Pasien Penyakit Mata

± 35 detik

Selamat pagi, Bapak/Ibu dosen dan teman-teman semua. Kami dari **Kelompok 3** akan mempresentasikan project berjudul **Sistem Manajemen Data Pasien Penyakit Mata**. Project ini kami buat dengan Python untuk menerapkan tiga konsep pemrograman berorientasi objek, yaitu **class dan object**, **enkapsulasi**, dan **inheritance**. *(tunjuk: angka di bawah judul)* Datanya bukan data karangan, melainkan data nyata **3.358 pasien** poli mata dari dataset ODIR-5K, dengan **7 kelompok penyakit** mata, yang kami susun dalam **9 class**. KLIK →

SLIDE 2 Sembilan bagian, sembilan pembicara

± 25 detik

Presentasi kami dibagi menjadi sembilan bagian, sesuai jumlah anggota. *(tunjuk: daftar)* Saya, Riswan, membuka dengan latar belakang dan struktur project. Yus Aidil menjelaskan dataset, Fahmi tentang class dan object, Diva tentang enkapsulasi, Rosa tentang inheritance, Sofiang tentang komposisi, Revalinda tentang class penyakit dan polimorfisme, Andi tentang program utama, lalu Alfira menunjukkan hasil program dan kesimpulan. KLIK →

SLIDE 3 Riswan

± 5 detik

Kita mulai dari bagian pertama, yaitu latar belakang dan struktur project. KLIK →

SLIDE 4 Data poli mata perlu dikelola dengan rapi

± 50 detik

(tunjuk: kartu Situasi) Situasinya begini. Di poli mata, kedua mata pasien difoto dengan kamera fundus, yaitu foto bagian dalam mata atau retina. Lalu dokter menuliskan diagnosisnya dalam bentuk **kata kunci**, misalnya moderate non proliferative retinopathy. *(tunjuk: kartu Masalah)* Masalahnya, datanya besar: **6.392 foto dari 3.358 pasien**. Satu pasien juga bisa punya lebih dari satu penyakit, dan data seperti ini rawan salah input, misalnya umur negatif atau penanda penyakit yang tidak cocok dengan diagnosisnya. *(tunjuk: kartu Pertanyaan)* Jadi pertanyaannya, bagaimana mengelola data ini agar **terstruktur, aman, dan mudah dicari**? *(tunjuk: kartu Jawaban)* Jawaban kami adalah program Python berbasis OOP: setiap pasien

menjadi object, setiap nilai dijaga oleh setter, dan jenis-jenis penyakit disusun sebagai **hierarki class**. KLIK →

SLIDE 5 Struktur project & cara menjalankan

± 50 detik

(tunjuk: daftar file) Kode program ada di folder **program**. Isinya **sepuluh file**: sembilan file class, satu class satu file, ditambah `main.py` sebagai program utama. Semua nama file memakai gaya snake_case, dan data aslinya ada di folder data, yaitu file **full_df.csv**. *(tunjuk: terminal)* Cara menjalankannya: masuk ke folder program dengan perintah `cd program`, lalu ketik **python main.py**. Program hanya memakai library bawaan Python, yaitu csv dan os, jadi tidak perlu instalasi tambahan. *(tunjuk: alur)* Alurnya ada empat langkah: membaca CSV, mengubah setiap baris menjadi object sambil melewati baris yang salah, menampilkan demo validasi, lalu membuka **menu interaktif** dengan enam pilihan. *(jeda)* Selanjutnya, **Yus Aidil** akan menjelaskan dataset yang kami pakai. KLIK →

Kalimat penutup bagianmu: “Selanjutnya, Yus Aidil akan menjelaskan dataset yang kami pakai.”

Persiapan tanya jawab

T **Kenapa memakai OOP, bukan program biasa (prosedural)?**

J Karena data pasien punya ciri (atribut) dan perilaku (method) yang jelas. Dengan OOP, setiap pasien menjadi object yang rapi, datanya dijaga lewat enkapsulasi, dan kodenya dipakai ulang lewat inheritance.

T **Apa itu foto fundus?**

J Foto bagian dalam belakang bola mata (retina, makula, dan saraf optik) yang diambil dengan kamera khusus. Banyak penyakit mata, seperti retinopati diabetik dan glaukoma, terlihat di foto ini.

T **Kenapa satu class satu file?**

J Supaya kode mudah dicari, dibaca, dan dibagi tugasnya. Setiap file hanya punya satu tanggung jawab.

T **Apakah harus masuk ke folder program dulu?**

J Disarankan. Tetapi `main.py` mencari file data berdasarkan lokasi file `main.py` sendiri, jadi perintah `python program/main.py` dari folder luar juga berjalan.

BAGIAN 2 | YUS AIDIL

Dataset ODIR-5K & Pemetaan Kolom

Slide 6–9 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 45 detik • 327 kata

Kata kunci: ODIR-5K · 6.392 baris · 3.358 orang · cataract · delapan kolom penanda · tidak ada atribut karangan · DO · retinopati diabetik · 1.105 pasien · 556 pasien · daftar penyakit

SLIDE 6 Yus Aidil

± 10 detik

Terima kasih, Riswan. Saya Yus Aidil, akan menjelaskan dataset yang kami pakai dan bagaimana kolomnya kami jadikan atribut. KLIK →

SLIDE 7 Data nyata dari rumah sakit, bukan karangan

± 1 menit

Data kami adalah **ODIR-5K**, singkatan dari Ocular Disease Intelligent Recognition. Data ini dikumpulkan oleh Shanggong Medical Technology dari beberapa rumah sakit di Tiongkok, dipakai pada kompetisi Peking University tahun 2019, dan kami unduh dari Kaggle. *(tunjuk: angka)* Isinya **6.392 baris**, dan setiap baris mewakili satu foto mata. Karena kedua mata pasien difoto, jumlah pasiennya **3.358 orang**, berumur 1 sampai 91 tahun dengan rata-rata 57,9 tahun. *(tunjuk: tabel)* Ini tiga baris pertama file aslinya. Pasien ID 0 berumur 69 tahun; mata kirinya **cataract** atau katarak, mata kanannya normal, sehingga penanda C bernilai 1. Pasien ID 2 punya dua diagnosis di mata kiri, dan penandanya D dan O. Jadi setiap baris berisi identitas pasien, diagnosis setiap mata, dan **delapan kolom penanda** dari dokter. KLIK →

SLIDE 8 Setiap kolom punya tempat di class

± 55 detik

Setiap kolom yang kami pakai kami jadikan atribut, dan **tidak ada atribut karangan** di luar data. *(tunjuk: tiga baris pertama tabel)* ID menjadi id_pasien di class Pasien, sedangkan umur dan jenis kelamin masuk ke class Orang. *(tunjuk: baris Fundus dan Keywords)* Nama foto dan diagnosis setiap mata disimpan di class PemeriksaanMata, satu object untuk mata kiri dan satu untuk mata kanan. *(tunjuk: baris penanda)* Delapan kolom penanda, N sampai O, digabung menjadi kode_diagnosis, misalnya **DO** untuk pasien ID 2. *(tunjuk: kolom aturan)* Di kolom kanan ada aturan yang dijaga setter, misalnya umur harus 1 sampai 120, dan penanda N yang artinya normal tidak boleh muncul bersama penyakit. *(tunjuk: catatan bawah)* Ada empat kolom yang tidak kami pakai, karena isinya hanya label per foto yang diturunkan dari kolom lain. KLIK →

SLIDE 9 Retinopati diabetik paling banyak ditemukan

± 40 detik

Ini sebaran penyakitnya, dihitung langsung oleh program kami. *(tunjuk: batang teratas)* Yang paling banyak adalah **retinopati diabetik**, yaitu kerusakan retina akibat diabetes, dengan **1.105 pasien**. Lalu kelainan lain 905 pasien, katarak 208, glaukoma 206, miopia patologis 171, degenerasi makula 163, dan retinopati hipertensi 103. *(tunjuk: angka di samping)* Sebanyak 1.080 pasien normal. Perlu dicatat, **556 pasien** punya lebih dari satu kelompok penyakit, jadi jumlah semua batang lebih besar dari jumlah pasien. Karena itu, satu pasien harus bisa menyimpan **daftar penyakit**, bukan hanya satu. *(jeda)* Selanjutnya, **Fahmi** akan menjelaskan class dan object. KLIK →

Kalimat penutup bagianmu: “Selanjutnya, Fahmi akan menjelaskan class dan object.”

Persiapan tanya jawab

T **Kenapa jumlah baris (6.392) berbeda dengan jumlah pasien (3.358)?**

J Karena satu baris mewakili satu foto. Pasien yang kedua fotonya ada muncul di dua baris dengan isi data pasien yang sama. Program menggabungkannya: 3.034 baris digabung.

T **Apakah datanya kalian ubah atau bersihkan dulu?**

J Tidak. File asli dipakai apa adanya; program yang memeriksa setiap nilainya. Hasilnya 0 baris dilewati, artinya data asli lolos semua aturan.

T **Kenapa kolom labels dan target tidak dipakai?**

J Isinya label per foto, bukan per pasien. Contohnya pasien ID 0 berpenanda C (katarak), tetapi baris foto mata kanannya berlabel ['N'] karena mata kanannya normal. Data pasien yang lengkap ada di 8 kolom penanda.

T **Apa yang dimaksud kelainan lain (O)?**

J Kelainan di luar enam kelompok utama. Di data ada 67 jenis, terbanyak macular epiretinal membrane (195 mata) dan drusen (179 mata).

BAGIAN 3 | FAHMI

Class & Object

Slide 10–13 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 45 detik • 326 kata

Kata kunci: cetakan · hasil cetakannya · atribut · method · 10.081 object · Orang · constructor · self · tampilkan_info · dua object · PasienMata · list

SLIDE 10 Fahmi

± 5 detik

Terima kasih, Yus. Saya Fahmi, akan menjelaskan konsep class dan object. KLIK →

SLIDE 11 Class adalah cetakan, object hasilnya

± 45 detik

Class bisa diibaratkan **cetakan** atau blueprint, sedangkan object adalah **hasil cetakannya**. *(tunjuk: kotak class)* Ini class PasienMata. Class menentukan data apa yang dimiliki pasien, disebut **atribut**, misalnya umur dan diagnosis, serta apa yang bisa dilakukan, disebut **method**, misalnya nama_penyakit. *(tunjuk: tiga kartu)* Dari satu cetakan itu, kita membuat banyak object dengan isi berbeda. Object pertama adalah pasien ID 0, perempuan 69 tahun dengan katarak. Object kedua pasien ID 1 yang normal. Object ketiga pasien ID 2 dengan retinopati diabetik dan kelainan lain. *(tunjuk: angka bawah)* Saat program berjalan, terbentuk **10.081 object**: 3.358 object pasien, 6.716 object pemeriksaan mata, dan 7 object jenis penyakit. KLIK →

SLIDE 12 Class Orang: cetakan paling dasar

± 50 detik

Ini kode class **Orang** di file `orang.py`. *(tunjuk: baris 6)* Baris 6, `class Orang`, fungsinya membuat class baru bernama Orang. *(tunjuk: baris 10)* Baris 10 berisi konstanta jenis kelamin yang valid, Male dan Female, sesuai isi data. *(tunjuk: baris 12)* Baris 12 adalah **constructor** `__init__`, yaitu method yang berjalan otomatis setiap kali object dibuat. Kata **self** berarti object itu sendiri, jadi setiap object menyimpan umurnya masing-masing. *(tunjuk: baris 15 dan 16)* Baris 15 dan 16 mengisi umur dan jenis kelamin. Nilainya dikirim lewat setter supaya langsung diperiksa; bagian ini nanti dijelaskan Diva. *(tunjuk: baris 42 sampai 45)* Lalu baris 42 membuat method **tampilkan_info**, dan baris 44 dan 45 mencetak umur dan jenis kelamin ke layar. Method ini nanti ditulis ulang oleh class turunannya. KLIK →

Penjelasan kode · orang.py baris 6–16

Baris	Kode	Fungsinya
6	`class Orang:`	Membuat class baru bernama Orang, cetakan paling dasar untuk data seseorang.
7	`"""Menyimpan data dasar seseorang: umur dan jenis kelamin."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).

Baris	Kode	Fungsinya
9	`# Nilai jenis kelamin yang ada di data (dicek dari nilai unik kolom Patient Sex)`	Komentar penjelas (tidak dijalankan Python).
10	`JENIS_KELAMIN_VALID = ("Male", "Female")`	Konstanta class: jenis kelamin yang sah, diambil dari nilai unik kolom Patient Sex.
12	`def __init__(self, umur, jenis_kelamin):`	**Constructor**: berjalan otomatis saat object dibuat, menerima umur dan jenis kelamin. `self` adalah object itu sendiri.
13	`# [ENKAPSULASI] Nilai diisi lewat setter, jadi langsung divalidasi`	Komentar penjelas (tidak dijalankan Python).
14	`# sebelum disimpan ke atribut private (self.__umur, self.__jenis_kelamin).`	Komentar penjelas (tidak dijalankan Python).
15	`self.umur = umur`	Mengisi `umur` lewat setter-nya, sehingga nilainya langsung diperiksa.
16	`self.jenis_kelamin = jenis_kelamin`	Mengisi `jenis_kelamin` lewat setter-nya, sehingga nilainya langsung diperiksa.

Penjelasan kode · orang.py baris 42–45

Baris	Kode	Fungsinya
42	`def tampilkan_info(self):`	Method `tampilkan_info()`: menampilkan data dasar. Nanti ditulis ulang (override) oleh class turunan.
43	`"""Menampilkan data dasar. Method ini di-override oleh class turunan."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
44	`print(f"Umur : {self.umur} tahun")`	Mencetak umur (dibaca lewat getter `self.umur`).
45	`print(f"Jenis Kelamin : {self.jenis_kelamin}")`	Mencetak jenis kelamin.

SLIDE 13 Satu baris CSV menjadi object

± 1 menit

Lalu bagaimana object dibuat dari file CSV? Ini fungsi `buat_pasien` di `main.py`. *(tunjuk: baris 93)* Fungsi ini menerima satu baris CSV dalam bentuk dictionary. *(tunjuk: baris 96 dan 97)* Baris 96 dan 97 membuat **dua object** PemeriksaanMata, untuk mata kiri dan mata kanan, berisi nama foto dan diagnosisnya. *(tunjuk: baris 98)* Baris 98 membuat object **PasienMata** dengan memanggil nama class seperti memanggil fungsi. *(tunjuk: baris 99 sampai 105)* Isinya: ID dan umur diubah dari teks menjadi angka oleh fungsi ke_bilangan_bulat, lalu jenis kelamin, dua object mata tadi, kode diagnosis dari kolom penanda, dan daftar penyakit hasil deteksi kata kunci. Di fungsi muat_data, baris 132, setiap object yang berhasil dibuat disimpan ke **list** daftar_pasien. Jadi satu baris CSV menghasilkan tiga object yang saling terhubung. *(jeda)* Selanjutnya, **Diva** akan menjelaskan bagaimana data di dalam object dijaga. **KLIK →**

Penjelasan kode · main.py baris 93–106

Baris	Kode	Fungsinya
93	`def buat_pasien(baris, daftar_jenis):`	Fungsi untuk mengubah satu baris CSV (dictionary) menjadi object `PasienMata`.
94	`"""[CLASS & OBJECT] Mengubah satu baris CSV menjadi object PasienMata."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
95	`# [KOMPOSISI] Object PemeriksaanMata dibuat dulu, lalu dimasukkan ke PasienMata`	Komentar penjelas (tidak dijalankan Python).

Baris	Kode	Fungsinya
96	`mata_kiri = PemeriksaanMata("kiri", baris["Left-Fundus"].strip(), baris["Left-Diagnostic Keywords"])`	Membuat object `PemeriksaanMata` mata kiri dari kolom Left-Fundus dan Left-Diagnostic Keywords.
97	`mata_kanan = PemeriksaanMata("kanan", baris["Right-Fundus"].strip(), baris["Right-Diagnostic Keywords"])`	Membuat object `PemeriksaanMata` mata kanan dari kolom Right-Fundus dan Right-Diagnostic Keywords.
98	`return PasienMata(`	Membuat object `PasienMata` dan langsung mengembalikannya, dengan isian:
99	`ke_bilangan_bulat(baris["ID"], "ID"),`	ID dari kolom ID, diubah menjadi bilangan bulat,
100	`ke_bilangan_bulat(baris["Patient Age"], "Umur"),`	umur dari kolom Patient Age,
101	`baris["Patient Sex"].strip(),`	jenis kelamin dari kolom Patient Sex,
102	`mata_kiri,`	object mata kiri,
103	`mata_kanan,`	object mata kanan,
104	`baca_kode_diagnosis(baris),`	kode diagnosis dari 8 kolom penanda,
105	`deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis),`	dan daftar penyakit hasil deteksi kata kunci.
106	`)`	Penutup tanda kurung yang dibuka di atasnya.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 132–132

Baris	Kode	Fungsinya
132	`daftar_pasien.append(buat_pasien(baris, daftar_jenis))`	Pasien baru: ubah baris menjadi object lewat `buat_pasien()`, lalu simpan ke list dengan `append`.

Kalimat penutup bagianmu: “Selanjutnya, Diva akan menjelaskan bagaimana data di dalam object dijaga.”

Persiapan tanya jawab

T Apa beda class dan object?

J Class adalah rancangannya, object adalah wujudnya. Class PasienMata hanya ditulis sekali, tetapi object-nya ada 3.358, satu untuk setiap pasien.

T Apa fungsi self?

J `self` menunjuk ke object yang sedang dipakai. Dengan `self`, setiap object menyimpan dan membaca datanya sendiri, misalnya umur pasien ID 0 tidak tercampur dengan pasien ID 1.

T Kapan __init__ dijalankan?

J Otomatis setiap kali object dibuat, misalnya saat `PasienMata(...)` dipanggil di `buat_pasien()`.

T Kenapa object disimpan di list?

J Supaya semua pasien bisa diproses bersama: diulang dengan `for`, dicari berdasarkan ID, dihitung ringkasannya, dan ditambah pasien baru dengan `append`.

BAGIAN 4 | DIVA NURISLAMIA AGFA

Enkapsulasi

Slide 14–18 • **Durasi** ± 4 menit • **Waktu baca** ± 3 menit 15 detik • 392 kata

Kata kunci: membungkus data · private · getter · setter · ValueError · 15 setter · salinan list · daftar_penyakit · validasi silang · sama · 3.358 pasien cocok 100 persen · minus 5 · tidak berhenti

SLIDE 14 Diva Nurislamia Agfa

± 5 detik

Terima kasih, Fahmi. Saya Diva, akan menjelaskan konsep enkapsulasi. KLIK →

SLIDE 15 Atribut private, dijaga getter & setter

± 55 detik

Enkapsulasi artinya **membungkus data** di dalam object dan membatasi cara mengubahnya. *(tunjuk: gambar kapsul)* Di project kami, semua atribut dibuat **private** dengan awalan dua garis bawah, misalnya `__umur`, sehingga tidak bisa diakses langsung dari luar class. Untuk membacanya kami memakai **getter** dengan dekorator @property, dan untuk mengubahnya kami memakai **setter**. Setter ini seperti petugas pendaftaran: setiap nilai diperiksa dulu, dan kalau salah, setter memunculkan **ValueError**, sehingga nilai itu tidak pernah tersimpan. *(tunjuk: daftar aturan)* Di project kami ada **15 setter**. Ini delapan aturan utamanya untuk data pasien, semuanya diambil dari nilai yang benar-benar ada di data, misalnya umur 1 sampai 120, jenis kelamin hanya Male atau Female, dan nama foto harus sesuai ID pasien. KLIK →

SLIDE 16 Setter memeriksa sebelum menyimpan

± 50 detik

Ini contoh enkapsulasi pada atribut umur di `orang.py`. *(tunjuk: baris 19 sampai 21)* Baris 19, `@property`, menjadikan method di bawahnya sebagai **getter**. Baris 21 mengembalikan nilai dari atribut private `self.__umur`. Jadi saat kita menulis `pasien.umur`, yang berjalan sebenarnya method ini. *(tunjuk: baris 23)* Baris 23, `@umur.setter`, menandai method **setter**, yang berjalan setiap kali ada perintah `pasien.umur = nilai`. *(tunjuk: baris 26 dan 27)* Baris 26 memeriksa nilainya: harus bilangan bulat antara 1 sampai 120. Kalau tidak, baris 27 memunculkan **ValueError** dengan pesan yang jelas. *(tunjuk: baris 28)* Hanya nilai yang lolos yang sampai ke baris 28 dan disimpan. *(tunjuk: kode bawah)* Satu lagi: getter daftar_penyakit di class PasienMata mengembalikan **salinan list**, jadi isi asli di dalam object tidak bisa diubah dari luar. KLIK →

Penjelasan kode · orang.py baris 19–28

Baris	Kode	Fungsinya
19	`@property`	Dekorator `@property`: method di bawahnya menjadi **getter**, sehingga nilainya dibaca seperti atribut.
20	`def umur(self):`	Getter `umur`: dijalankan saat kode membaca `objek.umur`.
21	`return self.__umur`	Mengembalikan nilai atribut private `__umur`.

Baris	Kode	Fungsinya
23	`@umur.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `umur` diubah.
24	`def umur(self, nilai):`	Setter `umur`: menerima nilai baru yang diperiksa sebelum disimpan.
25	`# Aturan: umur bilangan bulat 1 sampai 120 tahun`	Komentar penjelas (tidak dijalankan Python).
26	`if not isinstance(nilai, int) or nilai < 1 or nilai > 120:`	Validasi: menolak nilai yang bukan bilangan bulat (`int`) atau di luar 1 sampai 120.
27	`raise ValueError(f"Umur harus bilangan bulat 1 sampai 120, bukan {nilai}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
28	`self.__umur = nilai`	Nilai lolos pemeriksaan, lalu disimpan ke atribut private `__umur`.

Penjelasan kode · pasien_mata.py baris 71–73

Baris	Kode	Fungsinya
71	`@property`	Dekorator `@property`: method di bawahnya menjadi **getter**, sehingga nilainya dibaca seperti atribut.
72	`def daftar_penyakit(self):`	Getter `daftar_penyakit`: dijalankan saat kode membaca `objek.daftar_penyakit`.
73	`return list(self.__daftar_penyakit)  # kirim salinan agar isi asli tidak diubah dari luar`	Getter mengirim **salinan list**, sehingga list asli di dalam object tidak bisa diubah dari luar.

SLIDE 17 Validasi silang: penanda harus cocok

± 45 detik

> Setter yang paling penting di project kami ada di class PasienMata, yaitu setter **daftar_penyakit**. *(tunjuk: baris 77 sampai 83)* Pertama, setter memastikan isinya berupa list, dan setiap isinya adalah object PenyakitMata, lalu mengumpulkan kode penyakitnya. *(tunjuk: baris 86 dan 87)* Lalu ada **validasi silang**: penyakit yang dikenali dari kata kunci diagnosis harus **sama** dengan penanda yang diberi dokter di kolom D sampai O. *(tunjuk: baris 88)* Kalau berbeda, misalnya penanda katarak bernilai 1 tetapi diagnosisnya normal fundus, data itu ditolak. Kami sudah mengujinya ke semua data: **3.358 pasien cocok 100 persen**. Jadi aturan ini tidak membuang data asli, tetapi menjaga agar data baru tidak salah. KLIK →

Penjelasan kode · pasien_mata.py baris 75–90

Baris	Kode	Fungsinya
75	`@daftar_penyakit.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `daftar_penyakit` diubah.
76	`def daftar_penyakit(self, nilai):`	Setter `daftar_penyakit`: menerima nilai baru yang diperiksa sebelum disimpan.
77	`if not isinstance(nilai, list):`	Validasi: daftar penyakit harus berupa list.
78	`raise ValueError("Daftar penyakit harus berupa list")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
79	`kode_ditemukan = ""`	Menyiapkan teks kosong untuk mengumpulkan kode penyakit yang dikenali.
80	`for penyakit in nilai:`	Memeriksa setiap isi list.
81	`if not isinstance(penyakit, PenyakitMata):`	Setiap isi harus object `PenyakitMata` (atau turunannya).
82	`raise ValueError("Isi daftar penyakit harus object PenyakitMata")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.

Baris	Kode	Fungsinya
83	`kode_ditemukan += penyakit.kode`	Mengumpulkan kode penyakit, misalnya 'D' dan 'O' menjadi 'DO'.
84	`# Validasi silang: penyakit yang dikenali dari kata kunci diagnosis harus sama`	Komentar penjelas (tidak dijalankan Python).
85	`# dengan penanda yang diberi dokter (kolom D, G, C, A, H, M, O).`	Komentar penjelas (tidak dijalankan Python).
86	`kode_penanda = self.kode_diagnosis.replace("N", "")`	Mengambil kode penanda dokter tanpa huruf N.
87	`if sorted(kode_ditemukan) != sorted(kode_penanda):`	**Validasi silang**: membandingkan kedua kode tanpa memedulikan urutan huruf.
88	`raise ValueError(f"Penanda diagnosis '{self.kode_diagnosis}' tidak cocok dengan "`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
89	`f"kata kunci diagnosis (terbaca: '{kode_ditemukan or 'N'}')")`	Lanjutan pesan error: menampilkan kode yang terbaca dari kata kunci ('N' jika tidak ada).
90	`self.__daftar_penyakit = list(nilai)`	Menyimpan salinan list ke atribut private.

SLIDE 18 Demo: nilai salah ditolak, data tetap aman

± 40 detik

Sekarang kita lihat buktinya saat program dijalankan. *(tunjuk: kode)* Ini fungsi demo_validasi di `main.py`. Baris 148 sengaja mengisi umur dengan **minus 5** di dalam blok `try`. *(tunjuk: baris ValueError pertama di terminal)* Setter menolaknya dengan ValueError, lalu `except` di baris 149 menangkapnya, sehingga program **tidak berhenti**. *(tunjuk: umur sesudahnya)* Hasilnya, umur pasien ID 0 tetap 69 tahun. *(tunjuk: baris terakhir terminal)* Contoh kedua, kami mencoba membuat pasien berpenanda katarak, tetapi diagnosisnya normal fundus. Validasi silang menolaknya dengan pesan yang jelas. Jadi data yang salah tidak pernah masuk ke object. *(jeda)* Selanjutnya, **Rosa** akan menjelaskan konsep inheritance. KLIK →

Penjelasan kode · main.py baris 144–151

Baris	Kode	Fungsinya
144	`pasien = daftar_pasien[0]`	Mengambil object pasien pertama (ID 0) sebagai contoh.
145	`print(f"Umur pasien ID {pasien.id_pasien} sebelum diubah : {pasien.umur}")`	Mencetak umur sebelum dicoba diubah.
146	`print("Mencoba mengubah: pasien.umur = -5")`	Memberi tahu nilai yang akan dicoba.
147	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
148	`pasien.umur = -5  # tidak valid, umur harus 1 sampai 120`	Mengisi umur dengan -5 lewat setter; setter memunculkan `ValueError`.
149	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
150	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error dari setter.
151	`print(f"Umur pasien ID {pasien.id_pasien} sesudahnya : {pasien.umur} (tidak berubah)")`	Membuktikan umur tidak berubah, karena nilai salah tidak pernah disimpan.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 153–159

Baris	Kode	Fungsinya
153	`print("Mencoba membuat pasien berpenanda katarak (C), tetapi diagnosisnya normal")`	Memberi tahu contoh kedua: validasi silang.

Baris	Kode	Fungsinya
154	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
155	`mata_kiri = PemeriksaanMata("kiri", "9999_left.jpg", "normal fundus")`	Membuat object mata kiri dengan diagnosis normal.
156	`mata_kanan = PemeriksaanMata("kanan", "9999_right.jpg", "normal fundus")`	Membuat object mata kanan dengan diagnosis normal.
157	`PasienMata(9999, 50, "Male", mata_kiri, mata_kanan, "C", [])`	Membuat pasien berpenanda C (katarak) tanpa penyakit terdeteksi: ditolak setter `daftar_penyakit`.
158	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
159	`print(f"ValueError tertangkap: {error}")`	Mencetak pesan error dari setter.

Kalimat penutup bagianmu: “Selanjutnya, Rosa akan menjelaskan konsep inheritance.”

Persiapan tanya jawab

T **Kenapa memakai dua garis bawah, bukan satu?**

J Satu garis bawah hanya kesepakatan bahwa atribut itu internal. Dua garis bawah membuat Python mengganti namanya (name mangling), sehingga tidak bisa diakses langsung dengan nama aslinya.

T **Apakah atribut private benar-benar tidak bisa diakses?**

J Secara teknis masih bisa lewat nama `_Orang__umur`, tetapi itu melanggar aturan. Tujuan enkapsulasi adalah mencegah perubahan yang tidak sengaja dan memastikan semua perubahan lewat setter.

T **Kenapa constructor menulis self.umur, bukan self.__umur?**

J Supaya nilai awal juga melewati setter dan ikut divalidasi. Kalau langsung ke `self.__umur`, data salah dari CSV bisa lolos.

T **Kenapa memakai ValueError?**

J ValueError adalah exception bawaan Python untuk nilai yang tidak sesuai aturan. Pemanggil bisa menangkapnya dengan try-except, sehingga program tidak berhenti.

BAGIAN 5 | ROSA RAMADANI

Inheritance: Pewarisan Bertingkat

Slide 19–22 • **Durasi** ± 3 menit • **Waktu baca** ± 2 menit 25 detik • 291 kata

Kata kunci: mewarisi · bertingkat · method overriding · Pasien mewarisi Orang · constructor chaining · super() · Tiga class bekerja sama

SLIDE 19 Rosa Ramadani

± 5 detik

Terima kasih, Diva. Saya Rosa, akan menjelaskan konsep inheritance atau pewarisan. KLIK →

SLIDE 20 Orang → Pasien → PasienMata

± 50 detik

Inheritance artinya class anak **mewarisi** atribut dan method dari class induknya, jadi kode tidak perlu ditulis ulang. *(tunjuk: kotak Orang)* Di project kami ada pewarisan **bertingkat**. Paling atas class Orang, yang punya umur dan jenis kelamin. *(tunjuk: kotak Pasien)* Class Pasien mewarisi Orang dan menambah id_pasien. *(tunjuk: kotak PasienMata)* Lalu PasienMata mewarisi Pasien dan menambah data khusus poli mata: dua pemeriksaan mata, kode diagnosis, dan daftar penyakit. Karena bertingkat, object PasienMata otomatis punya umur, jenis kelamin, dan ID tanpa menulis ulang kodenya. *(tunjuk: tanda panah melingkar)* Ketiganya punya method tampilkan_info dengan isi berbeda; ini disebut **method overriding**. Selain itu ada satu keluarga class lagi, yaitu class penyakit, yang nanti dijelaskan Revalinda. KLIK →

SLIDE 21 super() menyambung class anak ke induk

± 45 detik

Ini class Pasien di `pasien.py`. *(tunjuk: baris 8)* Baris 8, `class Pasien(Orang):` nama class induk ditulis di dalam kurung, artinya **Pasien mewarisi Orang**. *(tunjuk: baris 11)* Constructor-nya menerima tiga data: ID, umur, dan jenis kelamin. *(tunjuk: baris 13)* Baris 13, `super().__init__(umur, jenis_kelamin)`, menjalankan constructor milik Orang. Ini disebut **constructor chaining**: umur dan jenis kelamin cukup diurus oleh Orang, termasuk validasinya. *(tunjuk: baris 15)* Pasien hanya mengurus atribut tambahannya, yaitu id_pasien. *(tunjuk: baris 30 sampai 33)* Lalu baris 30 menulis ulang method tampilkan_info. Baris 32 mencetak ID pasien, dan baris 33 memanggil tampilkan_info milik Orang lewat **super()**, sehingga umur dan jenis kelamin ikut tercetak tanpa menulis kode yang sama dua kali. KLIK →

Penjelasan kode · pasien.py baris 8–15

Baris	Kode	Fungsinya
8	`class Pasien(Orang):`	Membuat class `Pasien` yang **mewarisi** `Orang`; nama class induk ditulis di dalam kurung.
9	`"""Pasien rumah sakit (turunan dari Orang)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).

Baris	Kode	Fungsinya
11	`def __init__(self, id_pasien, umur, jenis_kelamin):`	Constructor Pasien: menerima ID, umur, dan jenis kelamin.
12	`# [INHERITANCE] Constructor chaining: umur & jenis kelamin diurus oleh Orang`	Komentar penjelas (tidak dijalankan Python).
13	`super().__init__(umur, jenis_kelamin)`	**Constructor chaining**: menjalankan constructor `Orang` untuk mengisi dan memvalidasi umur serta jenis kelamin.
14	`# [ENKAPSULASI] Diisi lewat setter agar tervalidasi`	Komentar penjelas (tidak dijalankan Python).
15	`self.id_pasien = id_pasien`	Mengisi `id_pasien` lewat setter-nya, sehingga nilainya langsung diperiksa.

Penjelasan kode · pasien.py baris 29–33

Baris	Kode	Fungsinya
29	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
30	`def tampilkan_info(self):`	**Method overriding**: menulis ulang `tampilkan_info()` milik `Orang`.
31	`"""Override tampilkan_info() milik Orang: tambah ID pasien."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
32	`print(f"ID Pasien : {self.id_pasien}")`	Mencetak ID pasien.
33	`super().tampilkan_info()  # memakai ulang tampilan umur & jenis kelamin dari Orang`	Memanggil `tampilkan_info()` milik `Orang` lewat `super()`, sehingga umur dan jenis kelamin ikut tercetak.

SLIDE 22 Tiga tingkat bekerja dalam satu tampilan

± 45 detik

Ini class PasienMata, tingkat ketiga. *(tunjuk: baris 12)* Baris 12 menuliskan PasienMata mewarisi Pasien. *(tunjuk: baris 20)* Di constructor-nya, baris 20 memanggil `super().__init__` dengan ID, umur, dan jenis kelamin. Rantainya berlanjut: constructor Pasien memanggil constructor Orang. *(tunjuk: baris 110 sampai 114)* Method tampilkan_info juga di-override. Baris 112 memanggil tampilkan_info milik Pasien, lalu baris 113 dan 114 menampilkan diagnosis kedua mata. *(tunjuk: terminal)* Hasilnya terlihat saat mencari pasien ID 188: baris ID berasal dari class Pasien, umur dan jenis kelamin dari class Orang, sedangkan data mata dan penyakit dari PasienMata. **Tiga class bekerja sama** dalam satu tampilan. *(jeda)* Selanjutnya, **Sofiang** akan menjelaskan konsep komposisi. **KLIK →**

Penjelasan kode · pasien_mata.py baris 12–12

Baris	Kode	Fungsinya
12	`class PasienMata(Pasien):`	Membuat class `PasienMata` yang mewarisi `Pasien` (tingkat ketiga: Orang → Pasien → PasienMata).

Penjelasan kode · pasien_mata.py baris 17–20

Baris	Kode	Fungsinya
17	`def __init__(self, id_pasien, umur, jenis_kelamin, mata_kiri, mata_kanan,`	Constructor PasienMata menerima tujuh data …
18	`kode_diagnosis, daftar_penyakit):`	… termasuk kode diagnosis dan daftar penyakit.
19	`# [INHERITANCE] Constructor chaining: Pasien -> Orang`	Komentar penjelas (tidak dijalankan Python).
20	`super().__init__(id_pasien, umur, jenis_kelamin)`	**Constructor chaining** ke `Pasien`, yang lalu meneruskan umur dan jenis kelamin ke `Orang`.

Penjelasan kode · pasien_mata.py baris 110–114

Baris	Kode	Fungsinya
110	`def tampilkan_info(self):`	**Method overriding**: menulis ulang `tampilkan_info()` milik `Pasien`.
111	`"""Override tampilkan_info() milik Pasien: tambah hasil pemeriksaan dan penyakit."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
112	`super().tampilkan_info()  # tampilkan ID, umur, jenis kelamin`	Memanggil versi `Pasien` (yang juga memanggil versi `Orang`): ID, umur, dan jenis kelamin.
113	`self.mata_kiri.tampilkan_info()`	Komposisi: object mata kiri menampilkan diagnosisnya sendiri.
114	`self.mata_kanan.tampilkan_info()`	Object mata kanan menampilkan diagnosisnya.

Kalimat penutup bagianmu: “Selanjutnya, Sofiang akan menjelaskan konsep komposisi.”

Persiapan tanya jawab

T **Apa beda inheritance dan komposisi?**

J Inheritance adalah hubungan 'adalah' (PasienMata adalah Pasien). Komposisi adalah hubungan 'memiliki' (PasienMata memiliki dua PemeriksaanMata).

T **Apa yang terjadi kalau super().__init__ tidak dipanggil?**

J Atribut umur dan jenis kelamin tidak pernah diisi, sehingga saat dibaca akan muncul AttributeError.

T **Apa itu method overriding?**

J Menulis ulang method milik induk dengan nama yang sama di class anak, supaya perilakunya disesuaikan. Contohnya `tampilkan_info()` di PasienMata menambah data mata dan penyakit.

T **Kenapa harus bertingkat, tidak langsung PasienMata(Orang)?**

J Pasien berlaku umum untuk semua pasien (punya ID), sedangkan PasienMata khusus poli mata. Kalau nanti ada poli lain, cukup membuat turunan baru dari Pasien.

BAGIAN 6 | SOFIANG

Komposisi: Pemeriksaan Mata

Slide 23–26 • **Durasi** ± 3 menit • **Waktu baca** ± 2 menit 30 detik • 298 kata

Kata kunci: komposisi · adalah · memiliki · 6.716 object PemeriksaanMata · _left.jpg · _right.jpg · isinstance · sesuai ID pasien · 567 pasien · bukan penyakit

SLIDE 23 Sofiang

± 5 detik

Terima kasih, Rosa. Saya Sofiang, akan menjelaskan komposisi pada class PemeriksaanMata. KLIK →

SLIDE 24 Inheritance “adalah”, komposisi “memiliki”

± 50 detik

Selain inheritance, kami juga memakai **komposisi**. Bedanya mudah diingat. *(tunjuk: kartu kiri)* Inheritance berarti hubungan **adalah**: PasienMata adalah seorang Pasien. Komposisi berarti hubungan **memiliki**. *(tunjuk: diagram)* Satu PasienMata memiliki dua object PemeriksaanMata, yaitu mata kiri dan mata kanan, serta daftar object penyakit. PemeriksaanMata tidak mewarisi Pasien, karena mata bukan pasien; mata adalah bagian dari data pasien. *(tunjuk: isi mata kiri dan kanan)* Contohnya pasien ID 188. Object mata kirinya berisi diagnosis laser spot dan retinopati diabetik proliferatif, sedangkan mata kanannya katarak. Karena dipisah per mata, program bisa tahu penyakit ada di mata kiri, mata kanan, atau keduanya. Totalnya ada **6.716 object PemeriksaanMata**, dua untuk setiap pasien. KLIK →

SLIDE 25 Object mata yang dimasukkan ikut diperiksa

± 50 detik

Komposisi di project kami juga dijaga validasi. *(tunjuk: kode atas)* Di PemeriksaanMata, setter nama_foto di baris 40 memastikan foto mata kiri berakhiran **_left.jpg** dan mata kanan **_right.jpg**. Baris 43 menentukan akhiran dari sisi matanya, dan baris 44 dan 45 menolak nama yang salah. *(tunjuk: kode bawah)* Di PasienMata ada method private `__cek_pemeriksaan`, dengan dua garis bawah, sehingga hanya dipakai di dalam class. Baris 48 memakai **isinstance** untuk memastikan yang dimasukkan benar-benar object PemeriksaanMata dengan sisi yang benar. Baris 50 memastikan nama fotonya **sesuai ID pasien**; misalnya pasien ID 7 harus punya foto 7_left.jpg. Jadi data mata pasien lain tidak bisa tertukar. KLIK →

Penjelasan kode · pemeriksaan_mata.py baris 40–46

Baris	Kode	Fungsinya
40	`@nama_foto.setter`	Dekorator setter: method di bawahnya dijalankan setiap kali `nama_foto` diubah.
41	`def nama_foto(self, nilai):`	Setter `nama_foto`: menerima nilai baru yang diperiksa sebelum disimpan.

Baris	Kode	Fungsinya
42	`# Aturan data: foto mata kiri berakhiran _left.jpg, mata kanan _right.jpg`	Komentar penjelas (tidak dijalankan Python).
43	`akhiran = "_left.jpg" if self.sisi == "kiri" else "_right.jpg"`	Menentukan akhiran nama foto yang benar berdasarkan sisi mata.
44	`if not isinstance(nilai, str) or not nilai.endswith(akhiran):`	Validasi: nama foto harus teks yang berakhiran sesuai sisi.
45	`raise ValueError(f"Nama foto mata {self.sisi} harus berakhiran {akhiran}, bukan '{nilai}'")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
46	`self.__nama_foto = nilai`	Nilai lolos pemeriksaan, lalu disimpan ke atribut private `__nama_foto`.

Penjelasan kode · pasien_mata.py baris 45–52

Baris	Kode	Fungsinya
45	`def __cek_pemeriksaan(self, nilai, sisi, akhiran):`	**Method private** (awalan __): hanya dipakai di dalam class untuk memeriksa object mata.
46	`"""Method private: pemeriksaan harus object PemeriksaanMata, sisinya benar,`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
47	`dan nama fotonya sesuai ID pasien (contoh: ID 7 -> 7_left.jpg)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
48	`if not isinstance(nilai, PemeriksaanMata) or nilai.sisi != sisi:`	Validasi: harus object `PemeriksaanMata` (dicek dengan `isinstance`) dan sisinya benar.
49	`raise ValueError(f"Mata {sisi} harus berupa object PemeriksaanMata sisi {sisi}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
50	`if nilai.nama_foto != f"{self.id_pasien}{akhiran}":`	Validasi: nama foto harus sesuai ID pasien, misalnya ID 7 → `7_left.jpg`.
51	`raise ValueError(f"Nama foto '{nilai.nama_foto}' tidak sesuai dengan ID pasien {self.id_pasien}")`	Memunculkan `ValueError` beserta pesan kesalahan, sehingga nilai yang salah tidak disimpan.
52	`return nilai`	Mengembalikan object yang sudah lolos pemeriksaan.

SLIDE 26 Diagnosis dibaca sesuai data aslinya

± 50 detik

Method di PemeriksaanMata dibuat berdasarkan keadaan data yang sebenarnya. *(tunjuk: baris 16)* Ternyata di data asli, **567 pasien** memakai tanda koma lebar dari huruf Tiongkok sebagai pemisah diagnosis, bukan koma biasa. Karena itu baris 16 menyimpan tanda koma lebar sebagai konstanta. *(tunjuk: baris 60 sampai 67)* Method daftar_kata_kunci di baris 60 mengganti koma lebar menjadi koma biasa, mengubah huruf menjadi kecil, lalu memecah diagnosis menjadi daftar kata kunci. *(tunjuk: baris 69 sampai 75)* Method catatan_foto memisahkan catatan yang **bukan penyakit**, misalnya lens dust, yaitu debu di lensa kamera. Catatan ini ada di 225 foto. Jadi catatan kualitas foto tidak salah dihitung sebagai penyakit. *(jeda)* Selanjutnya, **Revalinda** akan menjelaskan class-class penyakit mata. **KLIK →**

Penjelasan kode · pemeriksaan_mata.py baris 16–16

Baris	Kode	Fungsinya
16	`KOMA_LEBAR = "\uff0c"  # tanda koma lebar (, ) yang dipakai di data sebagai pemisah`	Konstanta koma lebar (，) berkode Unicode `\uff0c`, yang dipakai sebagai pemisah diagnosis pada sebagian data.

Penjelasan kode · pemeriksaan_mata.py baris 60–75

Baris	Kode	Fungsinya
60	`def daftar_kata_kunci(self):`	Method untuk memecah diagnosis menjadi list kata kunci.
61	`"""Memecah diagnosis menjadi daftar kata kunci berhuruf kecil."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
62	`teks = self.diagnosis.replace(PemeriksaanMata.KOMA_LEBAR, ",").lower()`	Mengganti koma lebar menjadi koma biasa, lalu mengubah semua huruf menjadi huruf kecil.
63	`hasil = []`	Menyiapkan list kosong `hasil`.
64	`for kata in teks.split(","):`	Memecah teks di setiap tanda koma, lalu mengulang setiap potongannya.
65	`if kata.strip() != "":`	Potongan yang kosong diabaikan.
66	`hasil.append(kata.strip())`	Menambahkan kata kunci (tanpa spasi di tepi) ke list `hasil`.
67	`return hasil`	Mengembalikan list kata kunci.
69	`def catatan_foto(self):`	Method untuk mengambil catatan kualitas foto yang bukan penyakit.
70	`"""Catatan kualitas foto (bukan penyakit), misalnya 'lens dust'."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
71	`hasil = []`	Menyiapkan list kosong `hasil`.
72	`for kata in self.daftar_kata_kunci():`	Mengulang setiap kata kunci diagnosis mata ini.
73	`if kata in PemeriksaanMata.BUKAN_PENYAKIT and kata != "normal fundus":`	Memilih kata yang termasuk `BUKAN_PENYAKIT`, kecuali 'normal fundus' (hasil normal, bukan catatan foto).
74	`hasil.append(kata)`	Menyimpan catatan foto ke list `hasil`.
75	`return hasil`	Mengembalikan list catatan foto.

Kalimat penutup bagianmu: “Selanjutnya, Revalinda akan menjelaskan class-class penyakit mata.”

Persiapan tanya jawab

T **Kenapa PemeriksaanMata tidak mewarisi Pasien?**

J Karena hubungannya bukan 'adalah'. Mata bukan pasien, tetapi milik pasien, jadi yang tepat adalah komposisi.

T **Apa itu koma lebar dan kenapa perlu ditangani?**

J Koma lebar (，) adalah tanda koma dari huruf Tiongkok. Di data asli, sebagian diagnosis memakai koma ini sebagai pemisah. Tanpa penanganan, dua diagnosis akan terbaca sebagai satu.

T **Apa itu lens dust?**

J Catatan bahwa ada debu di lensa kamera saat memotret. Ini catatan kualitas foto, bukan penyakit, jadi tidak dihitung sebagai penyakit.

T **Kenapa sisi diisi sebelum nama_foto di constructor?**

J Karena setter `nama_foto` memakai `self.sisi` untuk menentukan akhiran yang benar (_left.jpg atau _right.jpg).

BAGIAN 7 | REVALINDA

Hierarki Class Penyakit & Polimorfisme

Slide 27–30 • **Durasi** ± 3,5 menit • **Waktu baca** ± 2 menit 40 detik • 321 kata

Kata kunci: perilaku khusus · tingkat keparahan · 67 jenis · diurutkan dari yang paling berat · tidak disebutkan · polimorfisme · penyakit.tampilkan_info · bukan

SLIDE 27 Revalinda

± 5 detik

Terima kasih, Sofiang. Saya Revalinda, akan menjelaskan hierarki class penyakit mata dan polimorfisme. KLIK →

SLIDE 28 Satu class induk, empat turunan khusus

± 50 detik

Penyakit mata juga kami jadikan class. *(tunjuk: kotak induk)* Class induknya PenyakitMata, dengan atribut kode, nama, bagian mata, dan kata kunci, serta method ditemukan_di untuk mengecek apakah penyakit itu ada di diagnosis satu mata. *(tunjuk: empat kotak turunan)* Ada empat class turunan yang punya **perilaku khusus**. RetinopatiDiabetik bisa membaca **tingkat keparahan**; dari data ada 977 mata tingkat sedang dan 32 mata proliferatif. DegenerasiMakula membedakan jenis kering dan basah. Glaukoma membedakan diagnosis pasti dan dugaan. KelainanLain mengenali **67 jenis** kelainan di luar enam kelompok utama. *(tunjuk: baris bawah)* Sementara katarak, retinopati hipertensi, dan miopia patologis tidak butuh perilaku khusus, jadi cukup dibuat sebagai object dari class induk dengan data yang berbeda. KLIK →

SLIDE 29 RetinopatiDiabetik membaca tingkat keparahan

± 50 detik

Ini class RetinopatiDiabetik. *(tunjuk: baris 9)* Baris 9 menunjukkan class ini mewarisi PenyakitMata. *(tunjuk: baris 13 sampai 18)* Baris 13 sampai 18 berisi konstanta TINGKAT: pasangan kata kunci dokter dan nama tingkatnya, **diurutkan dari yang paling berat**, yaitu proliferatif, berat, sedang, dan ringan. *(tunjuk: baris 27)* Method tingkat_keparahan di baris 27 hanya ada di class ini. Baris 29 memakai method warisan ditemukan_di untuk mengecek apakah mata itu punya retinopati diabetik. *(tunjuk: baris 31 sampai 34)* Kalau ada, baris 31 sampai 34 mencari tingkatnya dari yang paling berat. Kalau dokter tidak menyebut tingkatnya, hasilnya **tidak disebutkan**. Di constructor-nya, class ini juga memanggil `super().__init__` dengan kode D dan kata kuncinya sendiri, dan meng-override tampilkan_info untuk menambahkan tingkat setiap mata. KLIK →

Penjelasan kode · retinopati_diabetik.py baris 9–9

Baris	Kode	Fungsinya
9	`class RetinopatiDiabetik(PenyakitMata):`	Membuat class `RetinopatiDiabetik` yang mewarisi `PenyakitMata`.

Penjelasan kode · retinopati_diabetik.py baris 13–18

Baris	Kode	Fungsinya
13	`TINGKAT = (`	Konstanta `TINGKAT`: pasangan (kata kunci dokter, nama tingkat), diurutkan dari yang paling berat.
14	`("proliferative diabetic retinopathy", "proliferatif"),`	Tingkat paling berat: proliferatif.
15	`("severe nonproliferative retinopathy", "berat"),`	Tingkat berat.
16	`("moderate non proliferative retinopathy", "sedang"),`	Tingkat sedang (ditulis dokter dengan spasi: 'non proliferative').
17	`("mild nonproliferative retinopathy", "ringan"),`	Tingkat ringan.
18	`)`	Penutup tanda kurung yang dibuka di atasnya.

Penjelasan kode · retinopati_diabetik.py baris 27–35

Baris	Kode	Fungsinya
27	`def tingkat_keparahan(self, pemeriksaan):`	Method khusus: membaca tingkat retinopati diabetik di satu mata.
28	`"""Tingkat retinopati diabetik di satu mata (dicari dari yang paling berat)."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
29	`if not self.ditemukan_di(pemeriksaan):`	Memakai method warisan `ditemukan_di()`; jika mata ini tidak punya retinopati diabetik …
30	`return "-"`	… kembalikan tanda '-'.
31	`for kunci, tingkat in RetinopatiDiabetik.TINGKAT:`	Mengulang tingkat, mulai dari yang paling berat.
32	`for kata in pemeriksaan.daftar_kata_kunci():`	Mengulang setiap kata kunci diagnosis mata ini.
33	`if kunci in kata:`	Jika kata kunci tingkat itu ditemukan …
34	`return tingkat`	… kembalikan nama tingkatnya (tingkat paling berat yang ditemukan).
35	`return "tidak disebutkan"`	Retinopati diabetik ada, tetapi dokter tidak menyebut tingkatnya.

Kode terkait (dibahas, tidak tampil di slide) · retinopati_diabetik.py baris 20–24

Baris	Kode	Fungsinya
20	`def __init__(self):`	Constructor tanpa parameter, karena data retinopati diabetik selalu sama.
21	`# [INHERITANCE] Constructor chaining dengan nilai khusus retinopati diabetik`	Komentar penjelas (tidak dijalankan Python).
22	`super().__init__("D", "Retinopati diabetik", "retina",`	Constructor chaining: mengisi atribut induk dengan kode D, nama, dan bagian mata (retina) …
23	`["nonproliferative retinopathy", "non proliferative retinopathy",`	… serta list kata kunci pengenalnya …
24	`"proliferative diabetic retinopathy", "diabetic retinopathy"])`	… (lanjutan list kata kunci).

Kode terkait (dibahas, tidak tampil di slide) · retinopati_diabetik.py baris 37–42

Baris	Kode	Fungsinya
37	`# ---------- [INHERITANCE] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
38	`def tampilkan_info(self, mata_kiri, mata_kanan):`	Override `tampilkan_info()` milik `PenyakitMata`.

Baris	Kode	Fungsinya
39	`"""Override: tambahkan tingkat keparahan di setiap mata."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
40	`super().tampilkan_info(mata_kiri, mata_kanan)`	Menjalankan versi `tampilkan_info()` milik induk dulu (nama penyakit dan mata yang terkena).
41	`print(f"  tingkat: kiri {self.tingkat_keparahan(mata_kiri)}, "`	Menambahkan tingkat keparahan mata kiri …
42	`f"kanan {self.tingkat_keparahan(mata_kanan)}")`	… dan mata kanan.

SLIDE 30 Polimorfisme: satu perintah, hasil berbeda

± 55 detik

Bagian terakhir saya adalah **polimorfisme**, artinya satu perintah yang sama bisa menghasilkan perilaku berbeda sesuai class object-nya. *(tunjuk: kode atas)* Di PasienMata, baris 119 sampai 121, program mengulang daftar penyakit pasien dan memanggil **penyakit.tampilkan_info** untuk setiap penyakit. Perintahnya sama, tetapi Python menjalankan versi method sesuai class object-nya. *(tunjuk: terminal)* Lihat pasien ID 188. Retinopati diabetik menampilkan tingkatnya: proliferatif di mata kiri. Katarak memakai versi class induk, jadi hanya nama dan mata yang terkena. Kelainan lain menampilkan jenisnya, yaitu laser spot. *(tunjuk: kode bawah)* Polimorfisme juga dipakai saat mendeteksi penyakit: di baris 48, KelainanLain menulis ulang ditemukan_di, karena kelainan lain dikenali dari kata kunci yang **bukan** milik enam kelompok lain. *(jeda)* Selanjutnya, **Andi** akan menjelaskan program utama. **KLIK →**

Penjelasan kode · pasien_mata.py baris 119–121

Baris	Kode	Fungsinya
119	`for penyakit in self.__daftar_penyakit:`	Mengulang setiap penyakit milik pasien.
120	`# [POLIMORFISME] Versi tampilkan_info() mengikuti class object penyakitnya`	Komentar penjelas (tidak dijalankan Python).
121	`penyakit.tampilkan_info(self.mata_kiri, self.mata_kanan)`	**Polimorfisme**: perintahnya sama, tetapi versi `tampilkan_info()` yang berjalan mengikuti class object penyakitnya.

Penjelasan kode · kelainan_lain.py baris 47–50

Baris	Kode	Fungsinya
47	`# ---------- [POLIMORFISME] Method overriding ----------`	Komentar pembatas bagian kode (tidak dijalankan Python).
48	`def ditemukan_di(self, pemeriksaan):`	**Override** `ditemukan_di()` milik induk (polimorfisme), karena cara mengenali kelainan lain berbeda.
49	`"""Override: kelainan lain ditemukan jika daftar_kelainan() tidak kosong."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
50	`return len(self.daftar_kelainan(pemeriksaan)) > 0`	Kelainan lain dianggap ditemukan jika `daftar_kelainan()` tidak kosong.

Kalimat penutup bagianmu: “Selanjutnya, Andi akan menjelaskan program utama.”

Persiapan tanya jawab

T **Apa itu polimorfisme?**

J Satu perintah yang sama menghasilkan perilaku berbeda sesuai class object-nya. Di project kami, `penyakit.tampilkan_info()` menampilkan tingkat untuk retinopati diabetik, jenis untuk kelainan lain, dan versi dasar untuk katarak.

T **Kenapa katarak tidak dibuat class sendiri?**

J Karena data katarak tidak punya informasi tambahan seperti tingkat atau jenis. Membuat class tanpa perilaku khusus hanya menjadi class kosong.

T **Bagaimana program tahu sebuah penyakit ada di satu mata?**

J Method `ditemukan_di()` mencocokkan kata kunci penyakit dengan daftar kata kunci diagnosis mata itu. Hasilnya lalu divalidasi silang dengan penanda dokter.

T **Kenapa tingkat keparahan dicari dari yang paling berat?**

J Satu mata bisa punya lebih dari satu kata kunci. Urutan dari yang paling berat memastikan tingkat tertinggi yang dilaporkan.

BAGIAN 8 | ANDI REZKY NABILA

Program Utama: Membaca CSV & Menu

Slide 31–34 • **Durasi** ± 4 menit • **Waktu baca** ± 3 menit 5 detik • 371 kata

Kata kunci: muat_data · csv.DictReader · append · tetap berjalan · 0 baris dilewati · menu interaktif · setter yang sama

SLIDE 31 Andi Rezky Nabila

± 5 detik

Terima kasih, Revalinda. Saya Andi, akan menjelaskan program utama di file `main.py`. KLIK →

SLIDE 32 muat_data() membaca CSV dengan aman

± 55 detik

Data dibaca oleh fungsi **muat_data**. *(tunjuk: baris 111 sampai 114)* Baris 111 sampai 114 menyiapkan list daftar_pasien untuk menyimpan object, dictionary isi_per_id untuk mengingat pasien yang sudah dibaca, list baris_dilewati untuk mencatat nomor baris yang salah, dan penghitung baris yang digabung. *(tunjuk: baris 116)* Baris 116 membuka file dengan `with`, sehingga file otomatis ditutup. Encoding utf-8-sig membuat file tetap terbaca walaupun pernah disimpan ulang lewat Excel. *(tunjuk: baris 118)* Baris 118 memakai **csv.DictReader** dari modul bawaan, sehingga setiap kolom dibaca dengan namanya, misalnya Patient Age. Pengaturan restval membuat kolom yang hilang terbaca sebagai teks kosong, jadi ikut ditolak validasi. *(tunjuk: baris 119 sampai 121)* Sebelum membaca data, baris 119 sampai 121 mengecek apakah semua kolom wajib ada di header. Kalau tidak ada, program memberi pesan bahwa format file tidak sesuai. KLIK →

Penjelasan kode · main.py baris 109–122

Baris	Kode	Fungsinya
109	`def muat_data(lokasi_file, daftar_jenis):`	Fungsi untuk membaca seluruh file CSV menjadi list object.
110	`"""Membaca CSV, menggabungkan baris milik pasien yang sama, dan mencatat baris tidak valid."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
111	`daftar_pasien = []`	List untuk menampung semua object `PasienMata`.
112	`isi_per_id = {}  # ID -> isi kolom pasien dari baris pertama yang valid`	Dictionary ID → isi kolom pasien dari baris pertama, untuk mengenali baris kedua pasien yang sama.
113	`baris_dilewati = []  # nomor baris CSV yang tidak valid`	List nomor baris CSV yang tidak valid.
114	`baris_digabung = 0  # baris foto mata kedua dari pasien yang sama`	Penghitung baris foto kedua yang digabung.
115	`# utf-8-sig: tetap aman jika CSV pernah disimpan ulang lewat Excel (ada BOM)`	Komentar penjelas (tidak dijalankan Python).
116	`with open(lokasi_file, mode="r", encoding="utf-8-sig", newline="") as file:`	Membuka file untuk dibaca; `with` menutup file otomatis. `utf-8-sig` aman untuk file yang pernah disimpan Excel.
117	`# restval="": kolom yang hilang dianggap teks kosong, jadi ikut ditolak validasi`	Komentar penjelas (tidak dijalankan Python).

Baris	Kode	Fungsinya
118	`pembaca = csv.DictReader(file, restval="")`	Membaca CSV sebagai dictionary per baris (kolom diakses dengan namanya). `restval=""` mengisi kolom yang hilang dengan teks kosong.
119	`for kolom in KOLOM_PASIEN:`	Mengecek setiap kolom wajib …
120	`if kolom not in (pembaca.fieldnames or []):`	… apakah ada di header file (`fieldnames`).
121	`raise ValueError(f"Kolom '{kolom}' tidak ada di file CSV")`	Kolom wajib tidak ada: hentikan pembacaan dengan `ValueError` (ditangkap di `main()`).
122	`for baris in pembaca:`	Mengulang setiap baris data di file.

SLIDE 33 Baris kembar digabung, baris salah dilewati

± 1 menit 5 detik

Ini isi perulangannya. Setiap baris diproses di dalam blok `try`. *(tunjuk: baris 124 sampai 126)* Karena satu pasien bisa muncul di dua baris, satu baris per foto, baris 126 mengecek apakah ID-nya sudah pernah dibaca. *(tunjuk: baris 128 sampai 131)* Kalau sudah, isinya harus sama persis dengan baris pertama; kalau berbeda, baris itu ditolak. Kalau sama, baris dihitung sebagai baris digabung, lalu `continue` melompat ke baris berikutnya. *(tunjuk: baris 132)* Baris baru diubah menjadi object oleh buat_pasien, lalu disimpan ke list dengan **append**. *(tunjuk: baris 134 sampai 137)* Kalau ada ValueError dari setter mana pun, `except` menangkapnya, mencatat nomor baris CSV-nya, dan mencetak peringatan. Program **tetap berjalan**. *(tunjuk: angka)* Hasilnya, dari 6.392 baris terbentuk 3.358 object pasien, 3.034 baris digabung, dan **0 baris dilewati**, karena data aslinya memang bersih. Kami juga sudah menguji dengan file rusak: sembilan jenis kerusakan, semuanya dilewati dengan pesan yang tepat. KLIK →

Penjelasan kode · main.py baris 123–137

Baris	Kode	Fungsinya
123	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
124	`isi = tuple(baris[kolom] for kolom in KOLOM_PASIEN)`	Mengambil isi 15 kolom pasien sebagai tuple, untuk dibandingkan dengan baris lain.
125	`id_teks = baris["ID"].strip()`	Mengambil ID pasien (masih berupa teks).
126	`if id_teks in isi_per_id:`	Jika ID ini sudah pernah dibaca, ini baris foto kedua pasien yang sama:
127	`# Baris foto mata kedua: isinya harus sama dengan baris pertama`	Komentar penjelas (tidak dijalankan Python).
128	`if isi != isi_per_id[id_teks]:`	isinya harus sama dengan baris pertama; jika berbeda …
129	`raise ValueError(f"Data pasien ID {id_teks} berbeda dengan baris sebelumnya")`	… baris ditolak dengan `ValueError`.
130	`baris_digabung += 1`	Jika sama, hitung sebagai baris yang digabung …
131	`continue`	Lewati sisa perulangan, lanjut ke data berikutnya.
132	`daftar_pasien.append(buat_pasien(baris, daftar_jenis))`	Pasien baru: ubah baris menjadi object lewat `buat_pasien()`, lalu simpan ke list dengan `append`.
133	`isi_per_id[id_teks] = isi`	Mengingat isi baris ini untuk ID tersebut.
134	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.
135	`# Baris yang melanggar aturan dilewati dan dicatat, program tetap berjalan`	Komentar penjelas (tidak dijalankan Python).
136	`baris_dilewati.append(pembaca.line_num)`	Mencatat nomor baris CSV yang sedang dibaca (`line_num`).

Baris	Kode	Fungsinya
137	`print(f"[PERINGATAN] Baris {pembaca.line_num} dilewati: {error}")`	Mencetak peringatan berisi nomor baris dan alasannya.

SLIDE 34 Tambah pasien memakai setter yang sama

± 1 menit

Program dilengkapi **menu interaktif** dengan enam pilihan. Di fungsi jalankan_menu, menu diulang dengan `while True` di baris 294 sampai pengguna memilih keluar. Salah satu menunya adalah tambah pasien baru. *(tunjuk: baris 224)* ID pasien baru dibuat otomatis, yaitu ID terbesar ditambah satu, supaya tidak ada ID ganda. *(tunjuk: baris 229 sampai 232)* Lalu pengguna mengisi umur, jenis kelamin, dan diagnosis kedua mata. Diagnosis langsung dibungkus menjadi object PemeriksaanMata. *(tunjuk: baris 233 sampai 236)* Penyakitnya dideteksi dari kata kunci, lalu kodenya digabung. *(tunjuk: baris 238)* Object PasienMata dibuat dengan **setter yang sama** seperti saat membaca CSV, jadi data dari keyboard juga diperiksa. *(tunjuk: baris 239 sampai 242)* Kalau ada yang salah, pasien tidak ditambahkan dan program kembali ke menu. Data baru disimpan di list selama program berjalan, jadi file CSV asli tidak berubah. *(jeda)* Selanjutnya, **Alfira** akan menunjukkan hasil program dan kesimpulan kami. KLIK →

Penjelasan kode · main.py baris 224–224

Baris	Kode	Fungsinya
224	`id_baru = max(pasien.id_pasien for pasien in daftar_pasien) + 1`	ID baru = ID terbesar + 1, supaya tidak ada ID ganda.

Penjelasan kode · main.py baris 228–242

Baris	Kode	Fungsinya
228	`try:`	`try`: menjalankan kode di bawahnya sambil berjaga-jaga jika muncul error.
229	`umur = ke_bilangan_bulat(input("Umur : "), "Umur")`	Meminta umur, lalu mengubahnya menjadi bilangan bulat.
230	`jenis_kelamin = input("Jenis kelamin (Male/Female) : ").strip()`	Meminta jenis kelamin (spasi di tepi dibuang).
231	`mata_kiri = PemeriksaanMata("kiri", f"{id_baru}_left.jpg", input("Diagnosis mata kiri : "))`	Meminta diagnosis mata kiri dan langsung membuat object `PemeriksaanMata`; nama foto dibuat dari ID baru.
232	`mata_kanan = PemeriksaanMata("kanan", f"{id_baru}_right.jpg", input("Diagnosis mata kanan : "))`	Sama untuk mata kanan.
233	`daftar_penyakit = deteksi_penyakit(mata_kiri, mata_kanan, daftar_jenis)`	Mendeteksi penyakit dari kata kunci diagnosis.
234	`kode = ""`	Teks kosong untuk menampung kode penyakit.
235	`for penyakit in daftar_penyakit:`	Mengulang penyakit yang terdeteksi …
236	`kode += penyakit.kode`	… dan menggabungkan kodenya.
237	`# [ENKAPSULASI] Data baru diperiksa oleh setter yang sama dengan data CSV`	Komentar penjelas (tidak dijalankan Python).
238	`pasien = PasienMata(id_baru, umur, jenis_kelamin, mata_kiri, mata_kanan, kode or "N", daftar_penyakit)`	Membuat object pasien; `kode or "N"` berarti jika tidak ada penyakit, kodenya N (normal). Semua setter ikut memeriksa.
239	`except ValueError as error:`	Menangkap `ValueError` dari blok `try`; pesannya disimpan di `error`, dan program tidak berhenti.

Baris	Kode	Fungsinya
240	`print(f"Data tidak valid: {error}")`	Menampilkan alasan data ditolak.
241	`print("Pasien baru tidak ditambahkan.")`	Memberi tahu pasien tidak jadi ditambahkan.
242	`return`	Keluar dari fungsi lebih awal.

Kode terkait (dibahas, tidak tampil di slide) · main.py baris 292–317

Baris	Kode	Fungsinya
292	`def jalankan_menu(daftar_pasien, daftar_jenis):`	Fungsi menu utama yang berulang sampai pengguna memilih Keluar.
293	`"""Menampilkan menu berulang-ulang sampai pengguna memilih Keluar."""`	Docstring: keterangan singkat isi class/method (tidak dijalankan sebagai perintah).
294	`while True:`	Perulangan tanpa batas; berhenti hanya lewat `break`.
295	`cetak_judul("MENU")`	Mencetak judul menu.
296	`print(f"1. Lihat {JUMLAH_DITAMPILKAN} data pertama")`	Mencetak pilihan 1.
297	`print("2. Cari pasien berdasarkan ID")`	Mencetak pilihan 2.
298	`print("3. Cari pasien berdasarkan penyakit")`	Mencetak pilihan 3.
299	`print("4. Tambah pasien baru")`	Mencetak pilihan 4.
300	`print("5. Tampilkan ringkasan")`	Mencetak pilihan 5.
301	`print("6. Keluar")`	Mencetak pilihan 6.
302	`pilihan = input("Pilih menu (1-6): ").strip()`	Membaca pilihan pengguna dari keyboard.
303	`if pilihan == "1":`	Pilihan 1 …
304	`menu_lihat_data(daftar_pasien)`	… menjalankan menu lihat data.
305	`elif pilihan == "2":`	Pilihan 2 …
306	`menu_cari_id(daftar_pasien)`	… menjalankan menu cari ID.
307	`elif pilihan == "3":`	Pilihan 3 …
308	`menu_cari_penyakit(daftar_pasien, daftar_jenis)`	… menjalankan menu cari penyakit.
309	`elif pilihan == "4":`	Pilihan 4 …
310	`menu_tambah_pasien(daftar_pasien, daftar_jenis)`	… menjalankan menu tambah pasien.
311	`elif pilihan == "5":`	Pilihan 5 …
312	`menu_ringkasan(daftar_pasien, daftar_jenis)`	… menjalankan menu ringkasan.
313	`elif pilihan == "6":`	Pilihan 6 …
314	`print("Terima kasih. Program selesai.")`	… mencetak salam penutup …
315	`break`	… lalu keluar dari perulangan `while`.
316	`else:`	Jika kondisi sebelumnya tidak terpenuhi, bagian di bawahnya yang dijalankan.
317	`print("Pilihan tidak tersedia. Masukkan angka 1 sampai 6.")`	Pilihan lain: cetak pesan, lalu menu tampil lagi.

> **Kalimat penutup bagianmu:** “Selanjutnya, Alfira akan menunjukkan hasil program dan kesimpulan kami.”

Persiapan tanya jawab

T **Apa fungsi try-except di muat_data()?**

J Menangkap ValueError dari baris yang melanggar aturan, supaya satu baris salah tidak menghentikan pembacaan seluruh file. Nomor barisnya dicatat agar bisa diperiksa.

T **Kenapa memakai DictReader, bukan csv.reader biasa?**

J DictReader membaca setiap baris sebagai dictionary, jadi kolom diambil dengan namanya, misalnya `baris['Patient Age']`. Lebih mudah dibaca dan tidak tergantung urutan kolom.

T **Kenapa pasien baru tidak disimpan ke file CSV?**

J Disengaja, agar data asli tetap utuh. Pasien baru tersimpan di list selama program berjalan.

T **Apa arti if __name__ == "__main__"?**

J Fungsi `main()` hanya dijalankan jika file dijalankan langsung dengan `python main.py`, bukan saat file di-import oleh file lain.

BAGIAN 9 | ALFIRA YUNITA SYAM

Hasil Program, Demo Menu & Kesimpulan

Slide 35–41 • **Durasi** ± 5 menit • **Waktu baca** ± 4 menit 10 detik • 496 kata

Kata kunci: 3.358 object PasienMata · menu · nama_penyakit · tidak ditemukan · 206 pasien · mata_terdampak · ID 4785 · 3.359 · tidak ditambahkan · retinopati diabetik · Katarak · class dan object · Enkapsulasi · Inheritance · sistem pengelola data

SLIDE 35 Alfira Yunita Syam

± 10 detik

Terima kasih, Andi. Saya Alfira, akan menunjukkan hasil program saat dijalankan, lalu menutup dengan kesimpulan. KLIK →

SLIDE 36 Program berjalan: 3.358 object siap dipakai

± 40 detik

Ini output asli saat program dijalankan, tidak kami karang. *(tunjuk: terminal kiri)* Program menampilkan nama file data, lalu **3.358 object PasienMata** berhasil dimuat, 3.034 baris digabung, dan 0 baris dilewati. Setelah demo validasi yang tadi dijelaskan Diva, muncul **menu** dengan enam pilihan. *(tunjuk: terminal kanan)* Pilihan 1 menampilkan 10 data pertama dalam bentuk tabel: ID, umur, jenis kelamin, dan penyakitnya. Misalnya pasien ID 0 katarak, pasien ID 1 normal, dan pasien ID 2 retinopati diabetik dengan kelainan lain. Kolom penyakit ini berasal dari method **nama_penyakit** di class PasienMata. KLIK →

SLIDE 37 Mencari pasien berdasarkan ID atau penyakit

± 45 detik

Pilihan 2 mencari pasien berdasarkan ID; pasien ID 188 tadi sudah ditunjukkan Rosa dan Revalinda. *(tunjuk: terminal kiri)* Kalau ID-nya tidak ada, misalnya ID 12, program menampilkan pesan **tidak ditemukan**. Nomor ID di data memang tidak berurutan, ada 1.427 nomor yang kosong. Kalau yang diketik bukan angka, misalnya abc, program tidak error, tetapi memberi pesan bahwa ID harus berupa angka. *(tunjuk: terminal kanan)* Pilihan 3 mencari berdasarkan penyakit. Saat memilih glaukoma, program menemukan **206 pasien** dan menampilkan sepuluh pertama, lengkap dengan mata yang terkena: mata kiri, mata kanan, atau kedua mata. Informasi ini berasal dari method **mata_terdampak** di class PenyakitMata. KLIK →

SLIDE 38 Data valid diterima, data salah ditolak

± 40 detik

Pilihan 4 menambah pasien baru. *(tunjuk: terminal atas)* Kami mencoba pasien perempuan berumur 63 tahun, mata kiri cataract dan mata kanan moderate non proliferative retinopathy. Program memberi **ID 4785** secara otomatis, mengenali dua penyakit, yaitu retinopati diabetik dan katarak, lalu jumlah pasien bertambah menjadi **3.359**. *(tunjuk: terminal kanan bawah)* Saat pasien 4785 dicari lewat pilihan 2, datanya muncul lengkap, termasuk

tingkat retinopati sedang di mata kanan. *(tunjuk: terminal kiri bawah)* Lalu kami mencoba data salah: jenis kelamin ditulis Laki-laki. Setter menolaknya, dan pasien **tidak ditambahkan**. Ini bukti enkapsulasi juga bekerja untuk data yang diketik pengguna. KLIK →

SLIDE 39 Ringkasan dihitung dari semua object

± 45 detik

Pilihan 5 menampilkan ringkasan yang dihitung dari semua object. *(tunjuk: bagian atas terminal)* Jumlahnya 3.358 pasien, 1.800 laki-laki dan 1.558 perempuan, rata-rata umur 57,9 tahun, 1.080 pasien normal, dan 556 pasien punya lebih dari satu kelompok penyakit. *(tunjuk: tabel)* Tabel di bawahnya menghitung setiap penyakit: jumlah pasien, rata-rata umur, dan berapa yang terkena di kedua mata. *(tunjuk: angka di kanan)* Ada beberapa temuan menarik. Penyakit terbanyak adalah **retinopati diabetik**, 1.105 pasien. **Katarak** punya rata-rata umur tertinggi, 66,4 tahun, sesuai dengan sifatnya yang berkaitan dengan usia. Dan retinopati hipertensi paling sering mengenai kedua mata: 90 dari 103 pasien. KLIK →

SLIDE 40 Tiga konsep OOP bekerja bersama

± 1 menit

Sebagai kesimpulan, *(tunjuk: kolom pertama)* **class dan object** kami terapkan dengan sembilan class, satu class satu file, dan lebih dari sepuluh ribu object saat program berjalan. *(tunjuk: kolom kedua)* **Enkapsulasi** menjaga semua atribut private lewat getter dan setter, termasuk validasi silang antara penanda dokter dan diagnosisnya. *(tunjuk: kolom ketiga)* **Inheritance** dipakai di dua keluarga class: Orang, Pasien, dan PasienMata, serta PenyakitMata dengan empat turunannya, lengkap dengan super dan method overriding. *(tunjuk: daftar file di bawah kolom)* Di bawah setiap kolom tertulis letaknya di kode, lengkap dengan nama file dan nomor baris, jadi mudah dicek. Ditambah komposisi dan polimorfisme, program ini membaca 6.392 baris data nyata dengan setiap nilainya diperiksa. *(tunjuk: catatan bawah)* Sebagai catatan, data ini hanya memuat penyakit yang terlihat di foto fundus, dan program ini adalah **sistem pengelola data**, bukan alat diagnosis. KLIK →

SLIDE 41 Terima kasih

± 10 detik

Sekian presentasi dari Kelompok 3. Terima kasih atas perhatian Bapak/Ibu dosen dan teman-teman. Kami persilakan jika ada pertanyaan.

Kalimat penutup bagianmu: “Sekian presentasi dari Kelompok 3.”

Persiapan tanya jawab

T **Apakah output di slide asli?**

J Ya. Semua output disalin otomatis dari hasil menjalankan `python main.py`, tidak diketik ulang.

T **Kenapa ID 12 tidak ada?**

J Nomor ID di data tidak berurutan. ID-nya 0–4784, tetapi pasiennya hanya 3.358, jadi ada 1.427 nomor yang kosong.

T **Apakah program ini bisa mendiagnosis penyakit mata?**

J Tidak. Diagnosis tetap dari dokter. Program mengelola, memeriksa, dan merangkum data diagnosis itu.

T **Apa kelebihan program ini?**

J Data nyata, setiap nilai divalidasi, baris salah dilewati tanpa menghentikan program, penyakit dikenali per mata, dan strukturnya mudah dikembangkan untuk penyakit baru.

SESI TANYA JAWAB

Siapa menjawab apa

Pertanyaan dijawab oleh anggota yang membawakan topiknya. Alfira (pembicara terakhir) mengulang pertanyaan, lalu mempersilakan anggota yang sesuai.

Topik pertanyaan	Dijawab oleh	Slide
Latar belakang, struktur folder, alur, dan cara menjalankan	**Riswan**	1–5
full_df.csv: sumber, kolom, pemetaan ke atribut, dan sebaran penyakit	**Yus Aidil**	6–9
Class Orang, constructor, self, dan pembuatan object di buat_pasien()	**Fahmi**	10–13
Atribut private, getter & setter, validasi silang, dan demo data tidak valid	**Diva Nurislamia Agfa**	14–18
Orang → Pasien → PasienMata, super(), dan method overriding	**Rosa Ramadani**	19–22
Class PemeriksaanMata, hubungan 'memiliki', dan validasi object	**Sofiang**	23–26
PenyakitMata dan 4 class turunan, method khusus, overriding, polimorfisme	**Revalinda**	27–30
main.py: muat_data(), penggabungan baris, try-except, menu interaktif	**Andi Rezky Nabila**	31–34
Output nyata program, ringkasan data, kesimpulan, dan catatan	**Alfira Yunita Syam**	35–41

Jika belum yakin dengan jawabannya

- Ulangi pertanyaannya dengan kalimat sendiri untuk memastikan maksudnya.
- Tunjukkan buktinya: buka slide bagian terkait (klik nama di slide pembagian materi), lalu tunjuk baris kodenya. Bisa juga membuka file `.py` di folder `program` dan menjalankan `python main.py`.
- Kalimat aman: “Terima kasih atas pertanyaannya. Izin kami tunjukkan bagian kodenya agar lebih jelas.”
- Jawaban lengkap setiap topik ada di bagian **Persiapan tanya jawab** setiap anggota.